

Lab 04 — Hands-on lab: build the API image

Goal: write the Dockerfile of a (fake) Spring Boot API yourself, and provoke each trap of the course — context, cache, `CMD` / `ENTRYPOINT`, shell form, `USER` — with a build engine that has no daemon.

Prerequisites — Labs 01 to 03 done.

Files provided (`files/`) - `Api.java` — a 30-line HTTP API with no dependency. It exposes `/` and `/actuator/health`, reads `APP_MESSAGE`, `APP_PROFILE`, `SERVER_PORT` from the environment, and handles `SIGTERM`. **You will never have to modify it:** the labs are about containers, not Java. - `construire-jar.sh` — compiles `Api.java` into `api.jar` inside a disposable container, without installing a JDK on your WSL.

Step 1 — Prepare the project

```
mkdir -p ~/labo-docker/04 && cd ~/labo-docker/04
cp <lab-path>/files/Api.java .
cp <lab-path>/files/construire-jar.sh . && chmod +x construire-jar.sh
./construire-jar.sh
```

Observe the download of `docker.io/library/eclipse-temurin:21-jdk` (once only, ~490 MB), then `api.jar` of about 2.4 KB.

Explanation. You have just used a container as a **disposable tool**: the compilation happened inside a full JDK, mounted on your folder, and nothing of it remains. That is already the idea of the multi-stage build of lab 05.

→ JAVA

`javac` compiles a `.java` into `.class` files (*bytecode*, independent of the machine); `jar` packs the `.class` files into a ZIP archive with a manifest saying which class to launch. The **JDK** (*Development Kit*) contains these tools; the **JRE** (*Runtime Environment*) only contains what is needed to run — hence its half size, and the choice we will make for the final image.

Step 2 — A first Dockerfile

Create `Dockerfile`:

```
FROM docker.io/library/eclipse-temurin:21-jre-alpine
WORKDIR /app
COPY api.jar /app/api.jar
EXPOSE 8080
ENTRYPOINT ["java", "-jar", "/app/api.jar"]
CMD ["--spring.profiles.active=prod"]
```

```
podman build -t api-lab:1.0 .
podman images --format 'table {{.Repository}}\t{{.Tag}}\t{{.Size}}' | grep -E 'api-lab|temurin'
```

Observe the steps `STEP 1/6` to `STEP 6/6`, each followed by an identifier `--> ...`, then `COMMIT` `api-lab:1.0` and `Successfully tagged localhost/api-lab:1.0`. The image weighs **209 MB** — for a

2.4 KB JAR. Most of it is the JRE.

```
podman run -d --name api -p 18080:8080 api-lab:1.0
curl -s localhost:18080/ ; echo
curl -s localhost:18080/actuator/health ; echo
podman logs api
podman port api
```

Observe `{"message":"Bonjour depuis l'API","profile":"default"}`, `{"status":"UP"}`, the lines Arguments recus : `--spring.profiles.active=prod` then `API demarree sur le port 8080 (profil default)`, and `8080/tcp -> 0.0.0.0:18080`.

Explanation. `EXPOSE 8080` published nothing: `-p 18080:8080` created the redirection. Check it: `podman run -d --name api2 api-lab:1.0` then `curl -m 2 localhost:18080/` fails... and remember that rootless, a `-p 80:8080` would be refused.

```
podman rm -f -t 0 api api2
```

→ **WINDOWS / WSL**

Open `http://localhost:18080/actuator/health` in your Windows browser: WSL relays the port. That is how you will test the Angular front end in lab 07.

Step 3 — The build context

```
mkdir -p ../common && echo "private-key" > ../common/secret.txt
printf 'FROM docker.io/library/alpine\nCOPY ../common/secret.txt /\n' > Dockerfile.out-of-context
podman build -f Dockerfile.out-of-context -t attempt .
```

Observe the failure: `Error: building at STEP "COPY ../common/secret.txt /": ... possible escaping context directory error: copier: stat: "/common/secret.txt": no such file or directory`.

Explanation. Buildah brought the path back *inside* the context (`/common/secret.txt` relative to the folder) and found nothing there. It is not a permissions problem: the file is outside the perimeter.

Now measure the cost of an unfiltered context:

```
mkdir -p node_modules && dd if=/dev/zero of=node_modules/big.bin bs=1M count=200 2>/dev/null
printf 'FROM docker.io/library/alpine\nCOPY . /src\n' > Dockerfile.all
time podman build -q -f Dockerfile.all -t api-lab:all .
podman images --format '{{.Repository}}:{{.Tag}} {{.Size}}' | grep all
```

Observe a build of about 1.7 s... and an image of **218 MB** for an 8.7 MB `alpine`: the 200 MB of `node_modules` are inside.

```
printf 'node_modules\n*.bin\nDockerfile.out-of-context\n' > .dockerignore
time podman build -q -f Dockerfile.all -t api-lab:all2 .
podman images --format '{{.Repository}}:{{.Tag}} {{.Size}}' | grep all
```

Observe 8.71 MB for `all2`, and a faster build.

Explanation. With Docker, those 200 MB would first have been **transferred** to the daemon (`transferring context`); Podman reads the folder in place, so the build seems "fast". But the image ships everything the `COPY . .` touches. `.dockerignore` acts **before** any instruction: without it, `node_modules` would go into the published image. Podman also accepts the name `.containerignore` — same content, same effect.

Step 4 — The cache, and the order of instructions

Add a simulated "dependencies" step. Replace your `Dockerfile` with:

```
FROM docker.io/library/eclipse-temurin:21-jre-alpine
WORKDIR /app
RUN echo "downloading dependencies..." && sleep 5
COPY api.jar /app/api.jar
ENTRYPOINT ["java", "-jar", "/app/api.jar"]
```

```
time podman build -t api-lab:2.0 .
time podman build -t api-lab:2.0 .
```

Observe a first build of about 6.4 seconds, then a second in 0.7 s, with `--> Using cache` under each step.

Modify the JAR and rebuild:

```
touch Api.java && ./construire-jar.sh >/dev/null
time podman build -t api-lab:2.1 .
```

Observe that the `RUN ... sleep 5` step still prints `--> Using cache`: only the `COPY` and what follows are replayed. 0.7 s.

Now invert the order — the `COPY` **before** the `RUN`:

```
FROM docker.io/library/eclipse-temurin:21-jre-alpine
WORKDIR /app
COPY api.jar /app/api.jar
RUN echo "downloading dependencies..." && sleep 5
ENTRYPOINT ["java", "-jar", "/app/api.jar"]
```

```
podman build -t api-lab:3.0 .
./construire-jar.sh >/dev/null && time podman build -t api-lab:3.1 .
```

Observe that the 5 seconds are **paid every time**: 6.3 s.

Explanation. There, in miniature, is the difference between a 40-second Maven build and a 6-minute one: the volatile `COPY` placed before the expensive step invalidates everything that follows.

Step 5 — CMD versus ENTRYPOINT

Rebuild the image of step 2 (it has an `ENTRYPOINT` and a `CMD`) and observe the combination:

```
timeout 5 podman run --rm api-lab:1.0 | head -2
timeout 5 podman run --rm api-lab:1.0 --debug | head -2
```

Observe in the first case `Arguments recurs : --spring.profiles.active=prod`, in the second `Arguments recurs : --debug`. The `ENTRYPOINT` (`java -jar ...`) stayed; only the `CMD` was replaced.

Compare with an image that only has a `CMD`:

```
printf 'FROM docker.io/library/eclipse-temurin:21-jre-alpine\nCOPY api.jar /app/api.jar\nCMD ["java","-jar","/app/api.jar"]\n' > D-cmd
podman build -q -f D-cmd -t api-lab:cmd .
podman run --rm api-lab:cmd sh -c 'echo "I replace everything"'
```

Observe that the API does **not start at all**: with a `CMD` alone, the argument replaces the whole command.

```
podman run --rm --entrypoint sh api-lab:1.0 -c 'echo "shell obtained despite ENTRYPOINT"'
```

Observe that `--entrypoint` is the escape hatch for debugging.

Explanation. `CMD` = replaceable default value, `ENTRYPOINT` = fixed program to which arguments are appended. The company pattern is `ENTRYPOINT` + `CMD` for default arguments.

Step 6 — The *shell* form, or the broken shutdown

This is the most important experiment of the lab. Build the **same** application with the *shell* form, on an **Ubuntu** base (the `21-jre` image without a suffix):

```
printf 'FROM docker.io/library/eclipse-temurin:21-jre\nCOPY api.jar /app/api.jar\nENTRYPOINT java -jar /app/api.jar\n' > D-shell-debian
podman build -q -f D-shell-debian -t api-lab:shell-debian .
podman run -d --name s-deb api-lab:shell-debian
sleep 3
podman exec s-deb ps -o pid,args | head -3
```

Observe:

```
PID COMMAND
1 /bin/sh -c java -jar /app/api.jar
2 java -jar /app/api.jar
```

The shell stayed PID 1. Stop:

```
time podman stop s-deb
podman inspect --format 'code={{.State.ExitCode}}' s-deb
podman logs s-deb | tail -2
```

Observe the warning `resorting to SIGKILL`, `real 0m10.8s`, `code=137`, and **no** "SIGTERM recu" message: the *shutdown hooks* did not run.

Compare with the `exec` form of step 2:

```
podman run -d --name s-exec api-lab:1.0
sleep 3 ; podman exec s-exec ps -o pid,args | head -3
time podman stop s-exec
podman inspect --format 'code={{.State.ExitCode}}' s-exec
podman logs s-exec | tail -2
```

Observe `1 java -jar /app/api.jar --spring.profiles.active=prod`, a stop in **0.14 s**, `code=143`, and the lines `SIGTERM recu : arret propre en cours...` then `API arretee proprement`.

Finally, the same *shell* form but on an **Alpine** base:

```
printf 'FROM docker.io/library/eclipse-temurin:21-jre-alpine\nCOPY api.jar\n/app/api.jar\nENTRYPOINT java -jar /app/api.jar\n' > D-shell-alpine
podman build -q -f D-shell-alpine -t api-lab:shell-alpine .
podman run -d --name s-alp api-lab:shell-alpine ; sleep 3
podman exec s-alp ps -o pid,args | head -3
time podman stop s-alp ; podman inspect --format 'code={{.State.ExitCode}}' s-alp
```

Observe that this time Java **is** PID 1 and the stop is clean (`143`).

Explanation. Busybox's shell (Alpine) replaces itself with the command when it is simple; Ubuntu's `dash` does not. **The same Dockerfile therefore has two behaviours depending on the base image.** That is exactly the kind of bug that works on the developer's workstation and breaks in production. The `exec` form removes the question.

```
podman rm s-deb s-exec s-alp
```

Step 7 — The entry script without `exec`

The most frequent case in companies:

```
printf '#!/bin/sh\nnecho "preparing..."&java -jar /app/api.jar\n' > entrypoint.sh
chmod +x entrypoint.sh
printf 'FROM docker.io/library/eclipse-temurin:21-jre-alpine\nCOPY api.jar /app/api.jar\nCOPY\nentrypoint.sh /entrypoint.sh\nENTRYPOINT ["/entrypoint.sh"]\n' > D-script
podman build -q -f D-script -t api-lab:script .
podman run -d --name s-script api-lab:script ; sleep 3
podman exec s-script ps -o pid,args | head -4
time podman stop s-script
podman inspect --format 'code={{.State.ExitCode}}' s-script
```

Observe `1 {entrypoint.sh} /bin/sh /entrypoint.sh` and `2 java -jar /app/api.jar`, then the 10 seconds and code `137` — **even on Alpine**.

Fix it by adding `exec`:

```
printf '#!/bin/sh\necho "preparing..."& nexec java -jar /app/api.jar&n' > entrypoint.sh
podman build -q -f D-script -t api-lab:script2 .
podman rm -f -t 0 s-script && podman run -d --name s-script api-lab:script2 ; sleep 3
podman exec s-script ps -o pid,args | head -3
time podman stop s-script ; podman inspect --format 'code={{.State.ExitCode}}' s-script
podman rm s-script
```

Observe that there is no shell any more, the stop is instant and the code is 143.

Step 8 — ARG is not a safe

```
printf 'FROM docker.io/library/alpine&nARG DB_PASSWORD=empty&nRUN echo "build with $DB_PASSWORD" > /trace.txt&nCMD ["cat","/trace.txt"]&n' > D-arg
podman build -q -f D-arg --build-arg DB_PASSWORD='Secr3t!' -t api-lab:arg .
podman image inspect --format '{{json .Config.Env}}' api-lab:arg
podman history --no-trunc api-lab:arg --format '{{.CreatedBy}}' | head -3
```

Observe that Config.Env only contains PATH (true, ARG does not become ENV)... but that podman history shows |1 DB_PASSWORD=Secr3t! /bin/sh -c echo "build with \$DB_PASSWORD" > /trace.txt.

Explanation. The secret is in the image, readable by anyone who has it. The right method comes in lab 08.

Step 9 — USER, its placement, and what it becomes rootless

```
printf 'FROM docker.io/library/eclipse-temurin:21-jre-alpine&nUSER 1000:1000&nWORKDIR /app&nRUN mkdir /data&nCOPY api.jar /app/api.jar&nENTRYPOINT ["java","-jar","/app/api.jar"]&n' > D-user-early
podman build -f D-user-early -t api-lab:user-early . 2>&1 | grep -iE 'permission|error'
```

Observe the failure: mkdir: cannot create directory '/data': Permission denied, then Error: building at STEP "RUN mkdir /data": exit status 1.

Put USER last:

```
printf 'FROM docker.io/library/eclipse-temurin:21-jre-alpine&nWORKDIR /app&nRUN mkdir /data && chown 1000:1000 /data&nCOPY --chown=1000:1000 api.jar /app/api.jar&nUSER 1000:1000&nENTRYPOINT ["java","-jar","/app/api.jar"]&n' > D-user-ok
podman build -q -f D-user-ok -t api-lab:user-ok .
podman run --rm --entrypoint id api-lab:user-ok
```

Observe uid=1000(...) gid=1000(1000) groups=1000(1000) : the application no longer runs as root inside the container.

Now see what that gives on the host:

```
podman run -d --name u api-lab:user-ok ; podman run -d --name r api-lab:1.0 ; sleep 1
podman top u user,huser,pid,hpid,comm
podman top r user,huser,pid,hpid,comm
podman rm -f -t 0 u r
```

Observe:

USER	HUSER	PID	HPID	COMMAND	<- u: USER 1000 in the image
1000	100999	1	12929	java	
USER	HUSER	PID	HPID	COMMAND	<- r: root in the image
root	1000	1	13037	java	

Explanation. `USER` applies to everything that follows, build **and** run time; you place it right before `ENTRYPOINT`, after preparing the files. Rootless, the container's "root" is already your user (`HUSER 1000`); the container's UID 1000, on the other hand, is mapped to `100999`, a UID from the reserved range of `/etc/subuid` that has *no* rights on your host. `USER` therefore remains useful: it strips the application of root privileges *inside* the container (image files, ports < 1024, `apk add`), and above all, the same image will one day run under Docker or Kubernetes, where root is root.

Clean-up

```
podman rm -f -t 0 api api2 2>/dev/null
podman rmi api-lab:1.0 api-lab:2.0 api-lab:2.1 api-lab:3.0 api-lab:3.1 \
    api-lab:cmd api-lab:shell-debian api-lab:shell-alpine \
    api-lab:script api-lab:script2 api-lab:arg api-lab:user-ok \
    api-lab:all api-lab:all2 2>/dev/null
podman images --format '{{.Repository}}:{{.Tag}}' | grep api-lab
podman rmi $(podman images --filter dangling=true -q) 2>/dev/null
rm -rf ~/labo-docker/04/node_modules ~/labo-docker/04/./common
```

Keep `~/labo-docker/04/api.jar` and `Api.java` : labs 05 to 09 reuse them. Also keep the images `eclipse-temurin:21-jre-alpine` and `eclipse-temurin:21-jdk`.

What you must be able to state now

- The context determines what is copyable; with Podman nothing is transferred, but `.dockerignore` remains indispensable for what ends up in the image.
- An invalidated instruction invalidates all the following ones: order decides build time.
- `EXPOSE` publishes nothing.
- `ENTRYPOINT` fixes the program, `CMD` gives replaceable arguments, `--entrypoint` allows debugging.
- The *shell* form can break clean shutdown — and its behaviour depends on the base image. You measured it: 0.14 s / code 143 versus 10 s / code 137.
- An entry script must end with `exec`.
- A `--build-arg` is visible in `podman history`.
- `USER` goes right before `ENTRYPOINT` — and keeps its meaning rootless.